中國信託交易 API 功能說明文件

版權所有，嘉實資訊股份有限公司

目錄

目錄.....2
版本修訂.....4
1. 目的：.....5
2. 元件使用說明：.....5
2.1. C# 撰寫說明.....5
2.2. 元件相關說明.....7
2.3. Python 撰寫相關說明.....7
3. API 元件功能說明：.....9
3.1. 設定相關功能.....10
3.1.1. 元件初始化(Init).....10
3.1.2. 結束使用元件(Fini).....10
3.1.3. 登入(Login).....11
3.1.4. 登出(Logout).....11
3.1.5. 連線(Connect).....12
3.1.6. 斷線(Disconnect).....12
3.1.7. 整體初始化(Init2).....13
3.1.8. 完整結束元件(Fini2).....13
3.1.9. 取得帳號資訊(GetAccount).....15
3.1.10. 設定 LOG 目錄(SetLogPath).....16
3.1.11. 設定下單類型(SetExecOrderType).....17
3.1.12. 設定交易單位(SetLotSizeData).....17
3.1.13. 設定 API 回傳類型(SetEchoType).....19
3.2. 交易相關功能.....20
3.2.1. 證券.....20
3.2.1.1. 證券新單(Stock_NewOrder).....21
3.2.1.2. 證券改單(Stock_ModifyOrder).....22
3.2.1.3. 證券刪單(Stock_CancelOrder).....24
3.2.1.4. 證券委託查詢(Stock_QueryOrder).....25
3.2.1.5. 證券成交查詢(Stock_QueryMatch).....25
3.2.1.6. 證券庫存查詢(Stock_QueryPosition).....26
3.3. 元件的非同步回傳格式.....27
3.3.1. C# 建立 Call Back Function 方式.....27
3.3.2. CallBack 事件分類.....27
3.4. 憑證元件.....29
3.4.1. 憑證簽章測試.....29

附錄 A. 錯誤代碼.....30
附錄 B. 主動回報回傳格式.....31
證券欄位定義.....31
附錄 C. 一般回傳格式.....32
證券新單回傳格式.....33
證券改單回傳格式.....33
證券刪單回傳格式.....33
證券委託查詢回傳格式.....35
證券成交查詢回傳格式.....37
證券庫存查詢回傳格式.....38
附錄 D. 帳號回傳格式.....40

版本修訂

修訂人	修正日期	dll 版本	說明
陳敏華	2018/03/13	V1.0.4.0	建立說明文件
蔣宜安	2020/10/26	V1.0.5.0	證券新增盤中零股 TT = 7
蔣宜安	2021/08/24	V1.0.6.0	把期權拿掉
田以諾	2025/4/15	V1.0.6.0	新增 Python 撰寫相關說明

1. 目的：

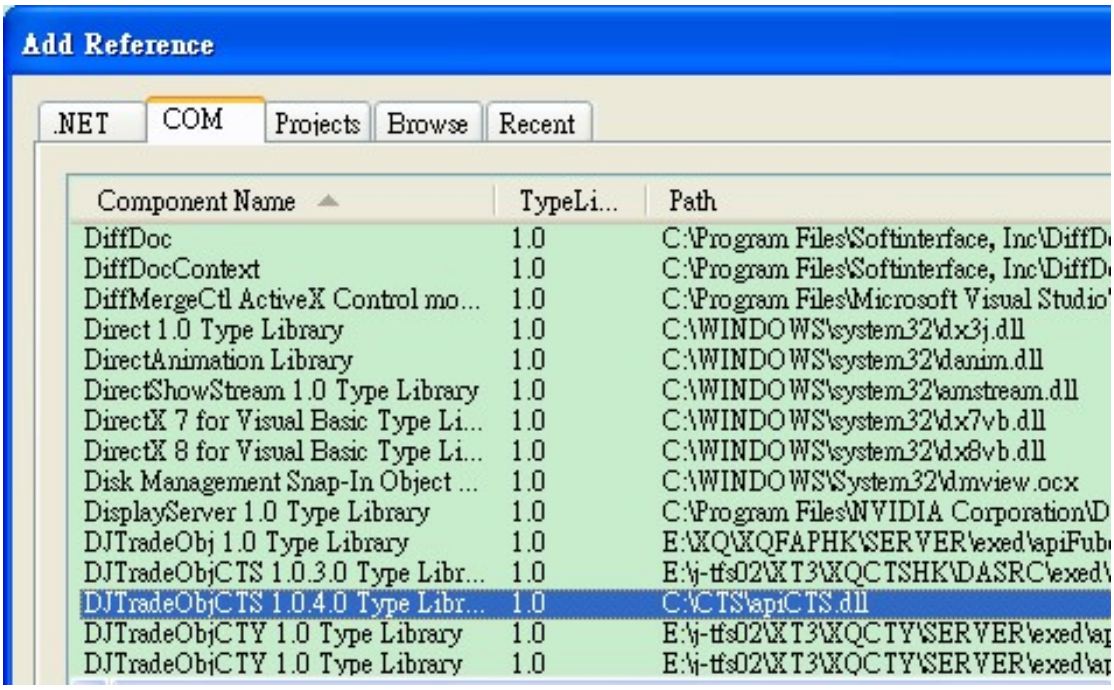
需安裝中國信託致富王，但使用時無需開啟中國信託致富王，提供交易 API 元件讓 Third Party 可以自行開發程式下單，使用 COM 方法，進行連接。

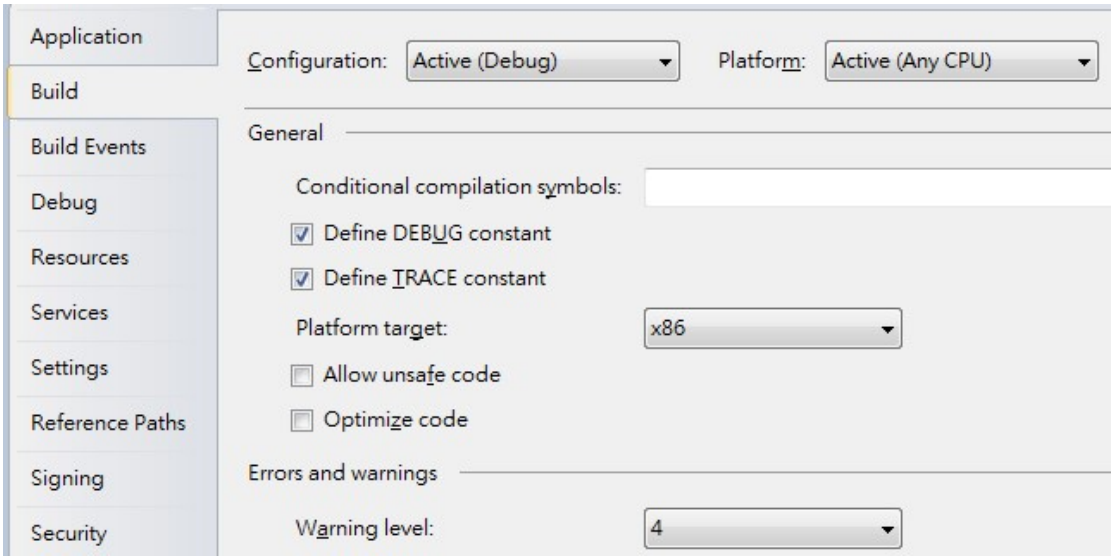
2. 元件使用說明：

安裝擁有交易 API 的中國信託致富王以後，將會擁有 apiCTS.dll 此檔案。安裝的同時會將此 COM 元件註冊。撰寫程式時，只要將此 COM 元件進行引用，就可以使用此元件，進行交易與查詢。

2.1.C# 撰寫說明

使用 C# 語言進行程式撰寫，必須先將 DJTradeObjCTS 1.0.4.0 Type Library 這個 COM 元件加入到 Reference 中，並且把 Properties 下 Platform target 設定成 x86。如下圖。



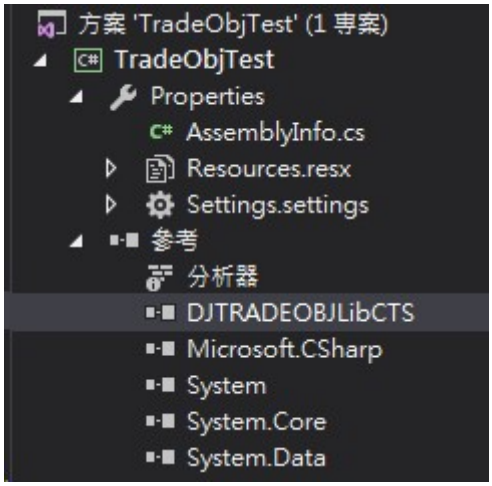


接著使用程式碼 using DJTRADEOBJLibCTS; 來使用 COM 元件。再使用 DJTRADEOBJLibCTS.TradeAppClass m_tradeApp; 宣告一個使用 interface 的物件。並且定義此物件 m_tradeApp = new TradeAppClass();。最後就可以使用元件所提供的 API。

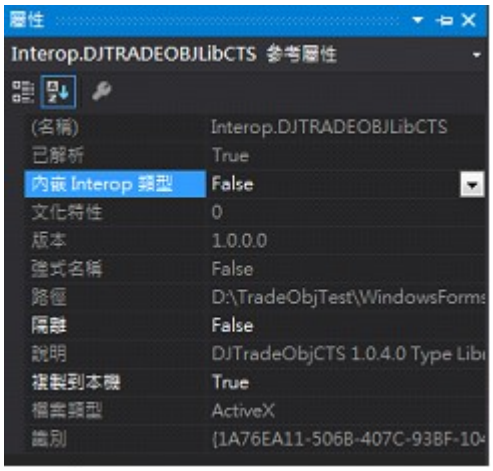
Ex: m_tradeApp.Init(strTradeDAS, out nErrCode, out sErrMsg);

※NET 4.0 以上之專案參考 COM 元件需注意是否內嵌 Interop 型別

步驟 1.點選 Reference 中 DJTRADEOBJLiBCTS 並按下” F4” ，進入屬性視窗。



步驟 2.將內嵌 Interop 類型修改為”False”。



2.2.元件相關說明

元件的變數型態 BSTR 對等 C# 的 String 型態。

2.3.Python 撰寫相關說明

使用 python 調用該 COM 元件，需安裝 32bit python，以及 pip install pywin32 套件。

具備事件的 COM 物件需先建立類別以及同名 method 來接收事件，並使用 win32com.client.DispatchWithEvents 來建立物件，如果接收事件的 method 須為動態物件 method，在建立之後重新指派即可，如下圖所示:

```
class EventHandler:
    def OnDataResponse(self,eventID,responseData):
        print(responseData)
        return

def InitTradeapp(self):
    #建立具備事件接收的com物件
    self.tradeApp = win32com.client.DispatchWithEvents(
        "DJTRADEOBJLibCTS.TradeApp",EventHandler)
    #重新指派視窗的method至OnDataResponse
    self.tradeApp.OnDataResponse=self.OnDataResponse
```

由於 python 不支援 out 參數，以下文件中的 out 參數會以 tuple 的形式依序回傳，以 Init 舉例:

C#

```
int nRet = m_tradeApp.Init(strTradeDAS, out nErrCode, out sErrMsg);
```

Python

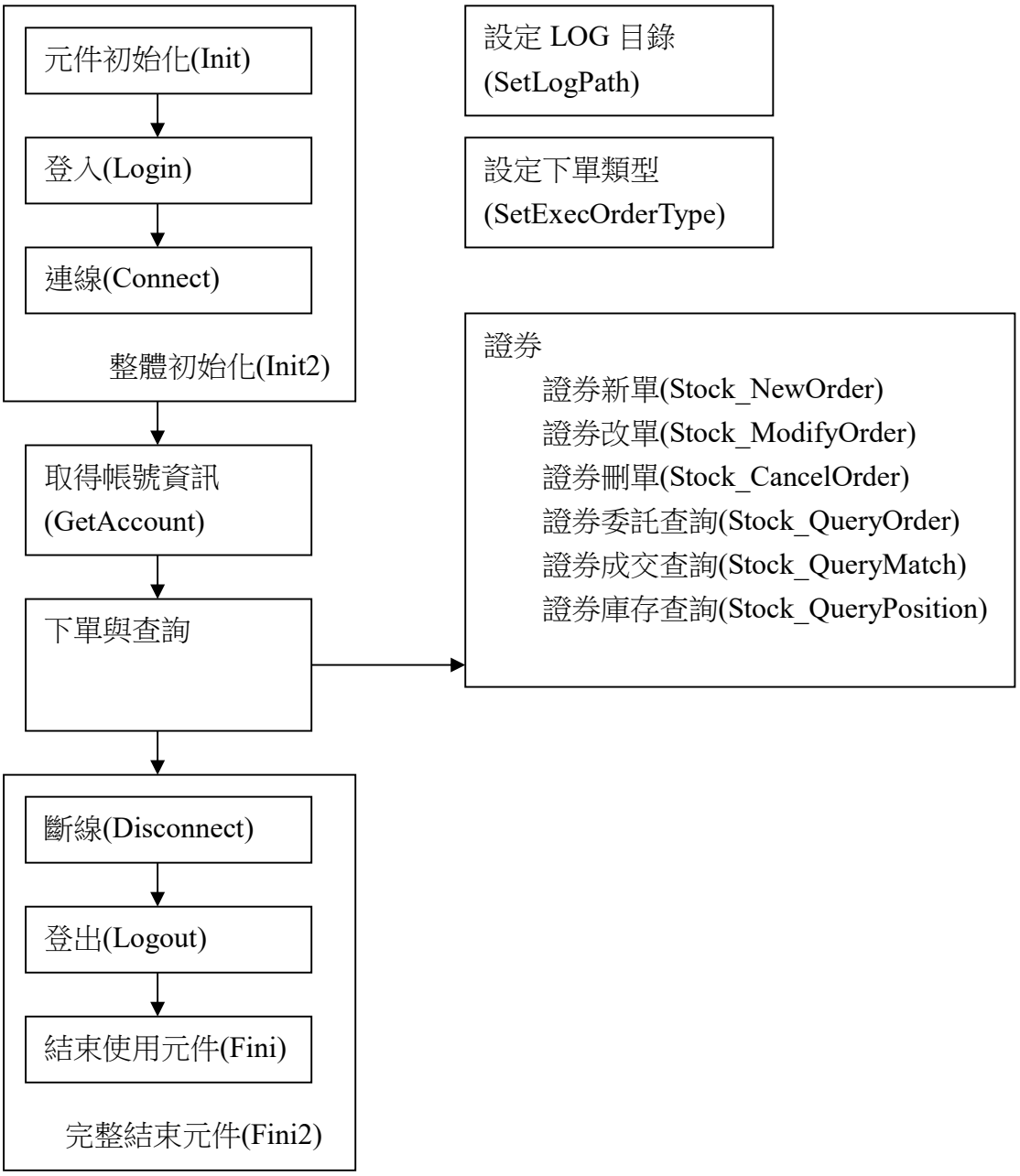
```
(temp, self.Errcode, self.Errmsg)=self.tradeApp.Init(strTradeDAS)
```

3. API 元件功能說明：

API 元件功能分為三大類：設定相關、交易相關、帳務相關。設定相關功能，主要負責，初始化，登入登出，主動回報連線，帳號取得，Log 目錄設定，Socket 下單設定。交易相關包涵，證券的新單、刪改單、委託查詢、成交查詢、庫存查詢、未平倉查詢。帳務相關則是帳務類別的查詢。

流程運作可分為三部份：初始化元件、交易與查詢、結束元件。初始化元件包涵 Init、Login、Connect 這三部份可以直接呼叫 Init2 一次完成。交易與查詢，則需要取得帳號並且呼叫提供的交易 API 與查詢 API。結束元件則需要 Disconnect、Logout、Fini 這三個動作，也可以直接呼叫 Fini2 會直接完成此三個步驟。

API 運作流程如下：



3.1.設定相關功能

初始元件呼叫順序必須依照：Init、Login，接著才可以呼叫 Connect、GetAccount、SetLogPath、SetExecOrder，結束時必須依照順序，Disconnect、Logout、Fini。

初始元件也可以直接呼叫 Init2，這個 Function 將會依照順序呼叫，Init、Login、Connect，一次做完。而結束時可呼叫 Fini2，將會依照順序呼叫 Disconnect、Logout、Fini。

3.1.1. 元件初始化(Init)

名稱：Init

說明：元件開始初始化，除了 SetLogPath 以外，其他的呼叫都必須在這個完成之後才可以呼叫。

參數：

```
Init(
    [in]BSTR sTradeDAS, //連線位置
    [out]long* pnErrCode,
    [out]BSTR* psErrMsg,
    [out, retval]long* pnRet
    );
```

回傳：1 成功, 0 失敗

C#
```
int nRet = m_tradeApp.Init(strTradeDAS, out nErrCode, out sErrMsg);
```

3.1.2. 結束使用元件(Fini)

名稱：Fini

說明：元件使用完，要結束程式時，使用這個可以釋放記憶體。完整結束

參數：無

回傳：無

C#
```
m_tradeApp.Fini();
```

3.1.3. 登入(Login)

名稱：Login

說明：Init 呼叫完畢後才可以執行此 API，功能為登入交易主機，取得交易設定，主動回報伺服器位址，以及交易帳號。

參數：

```
Login(
    [in]BSTR sUID,              // 身分證/統編
    [in]BSTR sPassword,         // 密碼
    [out]long* pnErrCode,       // 回傳錯誤碼
    [out]BSTR* psErrMsg,        // 回傳錯誤訊息
    [out, retval]long* pnRet
    );
```

回傳：1 成功, 0 失敗

C#

```
int nRet = m_tradeApp.Login(txtUID.Text, txtPwd.Text, out nErrCode, out sErrMsg)
```

3.1.4. 登出(Logout)

名稱：Logout

說明：向交易主機送出登出訊息。結束交易連線。

參數：

```
Logout(
    [in]BSTR sUID           // 身分證/統編

    );
```

回傳：無

C#

```
m_tradeApp.Logout(txtUID.Text);
```

3.1.5. 連線(Connect)

名稱：Connect

說明：主動回報連線，Socket 下單連線，必須在 Login API 呼叫完畢後才可執行。

參數：

Connect(

[out, retval]long* pnRet

);

回傳：1 成功, 0 失敗

C#

```
int nRet = m_tradeApp.Connect();
```

3.1.6. 斷線(Disconnect)

名稱：Disconnect

說明：登出主動回報連線。

參數：無

回傳：無

C#

```
m_tradeApp.Disconnect();
```

3.1.7. 整體初始化(Init2)

名稱：Init2

說明：一次完成 Init、Login、Connect 三個 API 所做的事情。可以搭配 Fini2 來執行。

參數：

```
Init2(
    [in]BSTR sTradeDAS,         // 連線主機 Ex. test.just.com/tradedas
    [in]BSTR sUID,              // 身分證/統編
    [in]BSTR sPassword,         // 密碼
    [out]long* pnErrCode,       // 錯誤代碼
    [out]BSTR* psErrMsg,        // 錯誤訊息
    [out, retval]long* pnRet
    );
```

回傳：1 成功, 0 失敗

C#

```
int nRet = m_tradeApp.Init2(strTradeDAS, txtUID.Text, txtPwd.Text, out nErrCode, out sErrMsg);
```

3.1.8. 完整結束元件(Fini2)

名稱：Fini2

說明：一次完成 Disconnect、Logout、Fini 三個 API 所做的事情。可搭配 Init2 來執行。

參數：

```
Fini2(
    [in]BSTR sUID           // 身分證/統編
    );
```

回傳：無

C#

```
m_tradeApp.Fini2(txtUID.Text);
```

3.1.9. 取得帳號資訊(GetAccount)

帳號取得有兩個方法，一個是先取得帳號數量，接著一筆一筆取出，可以依照不同的 Index 取得相對應的帳號。方法二是一次取得所有的帳號。可以直接在外面對帳號進行處理。

名稱：GetAccountCount
說明：取得所有帳號的數量，不分證券、期權。
參數：
GetAccountCount(
[out, retval]long* pnRet
);
回傳：整數數量

C#
```
int nCount = m_tradeApp.GetAccountCount();
```

名稱：GetAccount
說明：取得 idx 相對應的帳號。不分證券、期權。
參數：
GetAccount(
[in]long idx,
[out, retval]BSTR* psResult
);
回傳：請參照「附錄 D. 帳號回傳格式」

C#
```
int i;
string AccountData = m_tradeApp.GetAccount(i);
```

名稱：GetAccountList

說明：一次取出所有帳號資料。不分證券、期權。

參數：

```
GetAccountList(
    [out, retval]BSTR* psResult
    );
```

回傳：請參照「附錄 D. 帳號回傳格式」

C#

```
string AccountData = m_tradeApp.GetAccountList();
```

3.1.10. 設定 LOG 目錄(SetLogPath)

名稱：SetLogPath

說明：必須在呼叫 Init 以前設定，不然就會使用預設路徑(致富王安裝路徑)。

(此功能為工程師 Debug 使用，一般 USER 可以忽略此設定)

參數：

```
SetLogPath(
    [in]BSTR sLogPath,          // 絕對路徑
    [out, retval]BSTR* psError
    );
```

回傳：空字串成功，其他為錯誤訊息。

C#

```
string sLogPath;
string sResult = m_tradeApp.SetLogPath(sLogPath);
```

3.1.11. 設定下單類型(SetExecOrderType)

SetExecOrderType(long lType)

名稱：SetExecOrderType

說明：設定是走 Http 下單，或是 Socket 下單。

參數：

SetExecOrderType(

[in]long lType // 0: Auto, 1: HTTP, 2: Socket

);

回傳：無

C#

```
int nType = 0;
m_tradeApp. SetExecOrderType(nType);
```

3.1.12. 設定交易單位(SetLotSizeData)

名稱：SetLotSizeData

說明：交易單位非千股的商品，可以在此設定。

參數：

SetLotSizeData(

[in]BSTR sLotSizeData // Data: StockID1=LotSize1|StockID2=LotSize2

);

回傳：無

C#

```
string strLotSizeData = "0050=30010028=200";
m_tradeApp.SetLotSizeData(strLotSizeData);
```

3.1.13. 設定 API 回傳類型(SetEchoType)

名稱：SetEchoType

說明：API 呼叫的回傳類型分為兩類：同步與非同步。採用同步回傳時，呼叫會等 API 執行完後，回傳完整資料。採用非同步回傳時，當呼叫 API 後，COM 元件會產生一個 cookie，等執行完後，會執行 Call Back Function，將結果傳回。此時再依照 cookie 將執行的 API 與對應的結果做結合。API 呼叫類型分為三類：分別是 Order(下單)、QueryTrade(當日交易相關查詢)、QueryData(其他查詢)。可針對不同的類型做不同的回傳設定。

參數：

```
SetEchoType(
    [in]long lCmdType,      // 1: Order, 2: QueryTrade, 3: QueryData
    [in]long lEchoType      // 0: Sync, 1: ASync
    );
```

回傳：無

C#

```
int nCmdType = 1, nEchoType = 0;
m_tradeApp.SetEchoType(nCmdType, nEchoType);
```

3.2.交易相關功能

目前提供證券的交易模組，提供項目為新單、刪改單、當日委託查詢、當日成交查詢、即時庫存查詢、未平倉查詢。其他帳務查詢，必須提出新需求，才提供相關類型查詢。

3.2.1. 證券

使用參數說明：

證券帳號：券商分公司代號 - 證券帳號 Ex: 79Z-1234567

交易日期：合法交易日，格式為年月日 Ex: 20110313

股號：後台所需要的交易商品 Ex: 0050

強制查詢：nForceQuery，不使用元件快取資料，一定會向伺服器拿取資料

價格：可以有小數點的字串 Ex: 90.5

3.2.1.1. 證券新單(Stock_NewOrder)

名稱：Stock_NewOrder

說明：證券下單。根據不同下單類別型態，輸入不同參數。

參數：

```
Stock_NewOrder(
    [in]BSTR sAccountID,   // 證券帳號
    [in]long lTradeDate,   // 交易日期(西元年月日)
    [in]long nTT,          // 0: 普通/ 1: 零股/ 2: 盤後/ 5:興櫃/ 7:盤中零股
    [in]long nOT,          // 0: 現股/ 1:融資/ 2: 融券 / 16: 現沖先賣
    [in]long nBS,          // 1: 買進/ 2: 賣出
    [in]BSTR sStockID,     // 股號
    [in]long lQty,         // 普通&盤後(張數) / 零股(股數) / 興櫃(股數)
    [in]long nPT,          // 0: 限價/ 1: 漲停/ 2: 跌停/ 3: 平盤
    [in]BSTR sPrice,       // 價格
    [in]BSTR sBroker,      // 推薦券商 for 興櫃(保留未使用)
    [in]long nPayType,     // 0:餘額交割/ 1:逐筆交割 (for 興櫃) (保留未使用)
    [in]long nCond,        // Cond 委託條件 - 0: ROD, 1: FOK, 2: IOC
    [out, retval]BSTR* psResult
    );
```

回傳：請參照「附錄 C. 一般回傳格式的證券新單回傳格式」

C#

```
string strResult = m_tradeApp.Stock_NewOrder(szAccountID, m_nTradeDate, nTT, nOT, nBS, szStockID, nQty, nPT,
szPrice, szBroker, nPayType, nCond);
```

3.2.1.2. 證券改單(Stock_ModifyOrder)

名稱：Stock_ModifyOrder

說明：證券改單。

參數：

```
Stock_ModifyOrder(
    [in]BSTR sAccountID,  // 證券帳號
    [in]long nTradeDate,  // 交易日期(西元年月日)
    [in]long nTT,         // 0: 普通/ 1: 零股/ 2: 盤後/ 5:興櫃/ 7:盤中零股
    [in]long nOT,         // 0: 現股/ 1:融資/ 2: 融券
    [in]BSTR sOID,        // 委託單編號
    [in]BSTR sOrderNo,    // 委託書號
    [in]BSTR sStockID,    // 股號
    [in]long nBS,         // 1: 買進/ 2: 賣出
    [in]long nQty,        // 取消數量/取消後股數,
                          // 預約單/零股為取消後數量, 0=刪單
    [in]long nQcurrent,   // 有效張數
    [in]long nQmatch,     // 成交張數
    [in]long nPreOrder,   //(保留未使用，請帶 0)
    [in]long nType,       //(0:改單/ 2:改價)
    [in]BSTR sNewPrice,   // 新價位 (若 nNewPriceType =1,2,3,4 ,則
Price 給 0)
    [in]long nNewPriceType,   //新價位別(0: 限價/ 1:漲停/ 2:跌停/ 3:
 平盤/ 4:市價)
    [in]long nCond,   //新委託方式(0:ROD, 1:FOK, 2:IOC)(目前只支援非
市價 ROD 改非市價 ROD)
    [out, retval]BSTR* psResult
    );
```

回傳：請參照「附錄 C. 一般回傳格式的證券改單回傳格式」

C#

```
string strResult = m_tradeApp.Stock_ModifyOrder(szAccountID, m_nTradeDate, nTT, nOT,
                szOID, szOrderNo, txtStockID.Text, nBS, nQty, nQCurrent, nQMatch, nPreOrder, szNewPrice,
nNewPriceType, nCond);
```

3.2.1.3. 證券刪單(Stock_CancelOrder)

名稱：Stock_CancelOrder

說明：證券刪單

參數：

```
Stock_CancelOrder(
    [in]BSTR sAccountID, // 證券帳號
    [in]long nTradeDate,  // 交易日期(西元年月日)
    [in]long nTT,         // 0: 普通/ 1: 零股/ 2: 盤後/ 5:興櫃/ 7:盤中零股
    [in]long nOT,         // 0: 現股/ 1:融資/ 2: 融券
    [in]BSTR sOID,        // 委託單編號
    [in]BSTR sOrderNo,    // 委託書號
    [in]BSTR sStockID,    // 股號
    [in]long nBS,         // 1: 買進/ 2: 賣出
    [in]long nQty,        // 0=刪單
    [in]long nQcurrent,       // 有效張數
    [in]long nQmatch,     // 成交張數
    [in]long nPreOrder,       // (保留未使用，請帶 0)
    [in]BSTR sPrice, // 價位 (若 nPriceType =1,2,3,4 ,則 Price 給 0)
    [in]long nPriceType,  //價位別(0: 限價/ 1:漲停/ 2:跌停/ 3:平盤/ 4:
市價)
    [in]long nCond,  //委託方式(0:ROD, 1:FOK, 2:IOC)

    [out, retval]BSTR* psResult
    );
```

回傳：請參照「附錄 C. 一般回傳格式的證券刪單回傳格式」

C#

```
string strResult = m_tradeApp.Stock_CancelOrder(szAccountID, m_nTradeDate, nTT, nOT,
                szOID, szOrderNo, txtStockID.Text, nBS, nQty, nQCurrent, nQMatch, nPreOrder, sPrice,
nPriceType, nCond);
```

3.2.1.4. 證券委託查詢(Stock_QueryOrder)

名稱：Stock_QueryOrder

說明：證券委託查詢，可以使用 nForceQuery 設定是否使用快取資料。

參數：

```
Stock_QueryOrder(
    [in]BSTR sAccountID,  //  證券帳號
    [in]long nForceQuery,   // 0:user cache/ 1:force query
    [out, retval]BSTR* psResult
    );
```

回傳：請參照「附錄 C. 一般回傳格式的證券委託查詢回傳格式」

C#

```
string strResult = m_tradeApp.Stock_QueryOrder(szAccountID, nForceQuery);
```

3.2.1.5. 證券成交查詢(Stock_QueryMatch)

名稱：Stock_QueryMatch

說明：證券成交查詢，可以使用 nForceQuery 設定是否使用快取資料。

參數：

```
Stock_QueryMatch(
    [in]BSTR sAccountID,  //  證券帳號
    [in]long nForceQuery,   // 0:user cache/ 1:force query
    [out, retval]BSTR* psResult
    );
```

回傳：請參照「附錄 C. 一般回傳格式的證券成交查詢回傳格式」

C#

```
string strResult = m_tradeApp.Stock_QueryMatch(szAccountID, nForceQuery);
```

3.2.1.6. 證券庫存查詢(Stock_QueryPosition)

名稱：Stock_QueryPosition

說明：證券庫存查詢，可以使用 nForceQuery 設定是否使用快取資料。

參數：

```
Stock_QueryPosition(
    [in]BSTR sAccountID,  //  證券帳號
    [in]long nTradeDate,    //  交易日期(西元年月日)
    [in]long nForceQuery,   // 0:user cache/ 1:force query
    [out, retval]BSTR* psResult
    );
```

回傳：請參照「附錄 C. 一般回傳格式的證券庫存查詢回傳格式」

C#

```
string strResult = m_tradeApp.Stock_QueryPosition(szAccountID, m_nTradeDate, nForceQuery);
```

3.3.元件的非同步回傳格式

非同步回傳是當呼叫 API 的時候，最終結果不會直接從呼叫 API 的結果中得到。API 呼叫的結果會得到一個 cookie 值，等到 COM 元件執行此 Request 結束後，才會透過 Call Back Function 將結果傳回來。因此必須實做 Call Back Function。

Call Back Function 的 Interface 定義如下

OnDataResponse([in]long nEventID, [in]BSTR sResponseData);

3.3.1. C# 建立 Call Back Function 方式

可以使用 C# Delegate 的方式來撰寫。

```
m_tradeApp.OnDataResponse += new
_ITradeAppEvents_OnDataResponseEventHandler(m_tradeApp_OnDataResponse);
```

說明：

m_tradeApp.OnDataResponse：這個事件是 COM 元件所提供的事件 Interface。此名稱不可作修改。

_ITradeAppEvents_OnDataResponseEventHandler：此為 C# 經過處理的事件處理的 Class。此名稱不可作修改。

m_tradeApp_OnDataResponse：此為自訂的 Call Back Function 名稱。可以自行定義所需要的名稱。此 Function 的功能為處理回傳的資料。參數有兩個 int nEventID, string sResponseData。

3.3.2. CallBack 事件分類

COM 要求 Call Back Function 的參數有兩個。分別是 int nEventID 和 string sResponseData。EventID 是事件代碼，ResponseData 是此事件回傳的資料。

事件分類如下：

- 事件代碼：1
  事件名稱：主動回報連線狀態
  資料內容：AccType,Status (Ex:1.1)
  AccType: 1(證券)
  Status: 0(斷線), 1(連線)
- 事件代碼：100
  事件名稱：主動回報
  資料內容：請參照「附錄 B. 主動回報回傳格式」

- 事件代碼: 101
  事件名稱: 下單 回傳(非同步)
  資料內容: 請參照「附錄 C. 一般回傳格式」
- 事件代碼: 102
  事件名稱: 交易查詢 回傳(非同步)
  資料內容: 請參照「附錄 C. 一般回傳格式」
- 事件代碼: 103
  事件名稱: 帳務查詢 回傳(非同步)
  資料內容: 請參照「附錄 C. 一般回傳格式」
- 事件代碼: 202
  事件名稱: 資料改變通知

3.4.憑證元件

登入完成後，可以針對身分證，進行憑證元件設定。設定憑證的路徑和密碼。可以在下單前，進行憑證簽章測試。

3.4.1. 憑證簽章測試

名稱：CASignCheck

說明：憑證簽章測試，帳號是放入證券帳號，而非身分證。

參數：

```
CASignCheck (
    [in]BSTR sAccountID, // 帳號
    [out, retval]BSTR* psResult
    );
```

回傳：錯誤訊息，如果為空，則表示簽章成功。

```
C#
string strResult = m_tradeApp.CASignCheck("79u-0123456");
```

附錄 A. 錯誤代碼

錯誤代碼	代表意義
0	無錯誤
1	執行逾時
9000	未定義的錯
9001	系統內部錯
9002	不合法的參數
9003	未支援的功能
9100	網路錯誤
9200	結果錯誤

附錄 B. 主動回報回傳格式

回傳格式：<F0=...|F1=...|F2=...|...|Fn=...>

證券欄位定義

F0	帳號
F1	回報類別 03 證券
F2	OP
F3	Broker ID
F4	Account ID
F5	委託書號
F6	普通'0'/零股'2'/盤後'3'/興櫃'4'/盤中零股'7'
F7	現股'0'/融資'3'/融券'4'
F8	股票代號
F9	價位: 9(6)V9(2)
F10	限價'0'/漲停價'1'/跌停價'2'/平盤價'3'/市價'4'/超限價'5'
F11	B/S
F12	數量，改量：更改後數量，刪單：取消數量，均不含成交
F13	改量前(only valid for 改量)：改量前數量，均不含成交
F14	委託/成交回報日期
F15	委託/成交回報時間
F16	營業員
F17	資料來源
F18	交易所成交序號 9(8)
F19	網路序號 X(8)
F20	'1'餘額交割/'2'逐筆交割
F21	推薦券商代號
F22	智慧下單類型, 1:通知,2:一般單回報,3:智慧單回報
F23	智慧下單網路序號
F24	子帳號
F25	子帳號成交序號
F26	'1'：表成交回報時皆為股數, '0':表成交回報時根據 F4 決定
F27	保留
F28	委託條件: FOK'F'/IOC'I'/ROD'R'

F29	委託/成交回報時間至毫秒 (HHMMSS.fff)
<F0=20601-0101093|F1=03|F2=11|F3=9661|F4=0101093|F5=X0001|F6=0|F7=0|F8=1108 |F9=00000743|F10=0|F11=B|F12=00000001|F13=00000000| F14=20110411|F15=094922|F16=902|F17=1|F18= |F19=633350 |F20= |F21= |F22=2|F23=0000000000|F24= |F25= |F26=0|F27= >	

附錄 C. 一般回傳格式

回傳格式：

<rc=#code|cookie=#cookie|err=#errcode|msg=#message|count=#record_count>
<#record0><#record1><#record2>...

格式說明：

rc	1 表示成功, 0 表示失敗
cookie	Request 編號
err	錯誤碼
msg	訊息，可能是錯誤訊息或警示訊息
count	資料筆數
record	資料內容

資料內容格式：<#record> = <F0=...|F1=...|F2=...|...|Fn=...>

資料編碼處理：編碼為 Big5 (可包涵中文字)

編碼前	編碼後
"&"	"&"
"<"	"<"
">"	">"
"|"	"&bar;"
"="	"&equ;"

證券新單回傳格式

F0	1 預約單, 0 盤中單
F1	交易日期
F2	委託書號
F3	委託編號
F4	Mode
F5	OrderStatus
F6	Code
F7	CodeMsg
範例：<F0=0|F1=20110408|F2=X0001|F3=12345>	

請參照「3.2.1.1 證券新單」

證券改單回傳格式

F0	1 預約單, 0 盤中單
F1	委託書號
F2	交易日期
F3	委託編號
F4	Mode
F5	OrderStatus
F6	Code
F7	CodeMsg
範例：<F0=0|F1=20110408|F2=X0001|F3=12345|F4=1|F5=1|F6=|F7=>	

請參照「3.2.1.2 證券改單」

證券刪單回傳格式

F0	1 預約單, 0 盤中單
F1	交易日期
F2	委託書號
F3	委託編號
F4	Mode
F5	bValid
F6	Code
F7	CodeMsg

範例：<F0=0|F1=20110408|F2=X0001|F3=12345|F4=1|F5=1|F6=|F7=>

請參照「3.2.1.3 證券刪單」

證券委託查詢回傳格式

F0	szAccountID 證券帳號
F1	nTDate 交易日期
F2	nTT, 0: 普通/ 1: 零股/ 2: 盤後/ 5:興櫃/ 7:盤中零股
F3	nOT, 0: 現股/ 1:融資/ 2: 融券
F4	nBS, 買賣別 1: 買進/ 2: 賣出
F5	szStockID, 商品代號
F6	nPT, 0: 限價/ 1: 漲停/ 2: 跌停/ 3: 平盤
F7	szPrice, 價位
F8	nQty, 原委託張數/股數
F9	szOrderNo, 委託書號
F10	nQtyMatch, 已成交張數
F11	nQtyCancel, 已取消張數
F12	nOrderDate, 委託日期 (YYYYMMDD)
F13	nOrderTime, 委託時間 (HHMMSS)
F14	szStatusDesc, FALSE 失敗
F15	szOID, 編號
F16	nPreOrder, 預約單
F17	szAvgPrice, 均價
F18	nQcurrent, 有效張數
F19	nUpdateDate, 異動日期 (YYYYMMDD)
F20	nUpdateTime, 異動時間 (HHMMSS)
F21	nPayType, 0:餘額交割/ 1:逐筆交割
F22	szBroker, 推薦券商 for 興櫃
F23	nActionFlag
F24	CID 來源別
F25	OrderSource 委託來源
F26	nOrderStatus 1:錯誤單
F27	szCode
F28	szCodeMsg 錯誤訊息
F29	szExtraID (保留未使用)
F30	Cond 委託條件 - 0: ROD, 1: FOK, 2: IOC, 99: NULL
F31	nOrderTimeMS 委託時間到毫秒 (HHMMSSfff)
F32	nUpdateTimeMS 異動時間到毫秒 (HHMMSSfff)
範例：	

<F0=20601-0101093|F1=20110408|F2=0|F3=0|F4=1|F5=1108|F6=0|F7=7.38|F8=1000|F9=X0001|F10=0|F11=0|F12=20110408|F13=104132|F14=委託成功|F15=633330|F16=0|F17=0|F18=1000|F19=20110408|F20=104132|F21=0|F22=|F23=0|F24=5|F25=AP|F26=0|F27=0|F28=成功|F29=>

請參照「3.2.1.4 證券委託查詢」

證券成交查詢回傳格式

F0	strTradeID
F1	strTradeDate 交易日期
F2	nTT, 0: 普通/ 1: 零股/ 2: 盤後/ 5:興櫃/ 7:盤中零股
F3	strStockID, 商品代號
F4	nBS, 買賣別 1: 買進/ 2: 賣出
F5	nOT, 0: 現股/ 1:融資/ 2: 融券
F6	strPrice, 價位
F7	nQty, 原委託張數/股數
F8	strAmount, 價金
F9	strOrderNo, 委託書號
F10	strMatchDate, 成交日期 (YYYYMMDD)
F11	strMatchTime, 成交時間 (HHMMSS)
F12	strFee, 手續費
F13	strTax, 交易稅
F14	strOID, 編號
F15	strOMID, 交易編號
F16	strStockName, 股票名稱
F17	strOrderSource, 委託來源
F18	cCid, 下單管道
F19	Cond 委託條件(保留未使用)
F20	strMatchTimeMS, 成交時間到毫秒 (HHMMSSfff)
範例： <F0=20601-0101093|F1=20110408|F2=0|F3=1108|F4=1|F5=0|F6=7.38|F7=1|F8=7.38|F9=X0002|F10=20110408|F11=105032|F12=10|F13=2|F14=633335|F15=2752|F16=|F17=|F18=>	

請參照「3.2.1.5 證券成交查詢」

證券庫存查詢回傳格式

F0	nTT, 0: 普通/ 1: 零股/ 2: 盤後/ 5:興櫃/ 7:盤中零股
F1	strStockID, 商品代號
F2	集保昨日庫存
F3	集保今日庫存
F4	集保今日買進委託
F5	集保今日買進成交
F6	集保今日賣出委託
F7	集保今日賣出成交
F8	調整
F9	融資昨日庫存
F10	融資今日庫存
F11	融資今日買進委託
F12	融資今日買進成交
F13	融資今日賣出委託
F14	融資今日賣出成交
F15	調整
F16	融券昨日庫存
F17	融券今日庫存
F18	融券今日買進委託
F19	融券今日買進成交
F20	融券今日賣出委託
F21	融券今日賣出成交
F22	調整
F23	strCost, 成本
F24	strStockName, 股票名稱
F25	strRefPrice, 昨收
F26	strAMT, 昨收 * 昨日庫存
F27	strCRAmount, 融資金額
F28	strDBAmount, 融券保證金
F29	strDBGuarantee, 融券擔保金
範例： <F0=0|F1=2330|F2=1000|F3=1000|F4=0|F5=0|F6=0|F7=0|F8=0|F9=0|F10=0|F11=0|F12=0|F13=0|F14=0|F15=0|F16=0|F17=0|F18=0|F19=0|F20=0|F21=0|F22=0|F23=0|F24=台積電|F25=|F26=|F27=|F28=|F29=>	

請參照「3.2.1.6. 證券庫存查詢」

附錄 D. 帳號回傳格式

回傳格式：<#account0><#account1><#account2>...

帳號格式：<#account> = <ID=...|Name=...|UID=...|Type=...>

ID	證券帳號
Name	使用者名稱
UID	身份證帳號
Type	1 證券
Credit	0 無資券權限, 1 有資券權限
SFID	保留未使用
CanHedge	0 不可現股當沖, 1 可先買後賣,2 可先買後賣和先賣後買
範例： <ID=79Z-1234567|Name=證-安和 1234567-陳XX|UID=A123456789|Type=1|Credit=1|SFID=|CanHedge=2 >	